



Module 2. Convolutional Neural Networks

Concepts – Recap
Example Architectures
Visualizing Trained CNNs
Object Detection
Image Segmentation
Transfer Learning



Understanding Concepts – Recap and expansion

Concepts from last class

At this point all the following terms should be understandable:

- Receptive field vs. Filter
- Filter/Kernel
- Filter vs. Feature Map
- Feature map vs. Activation map
- Convolution
- Convolutional Layer

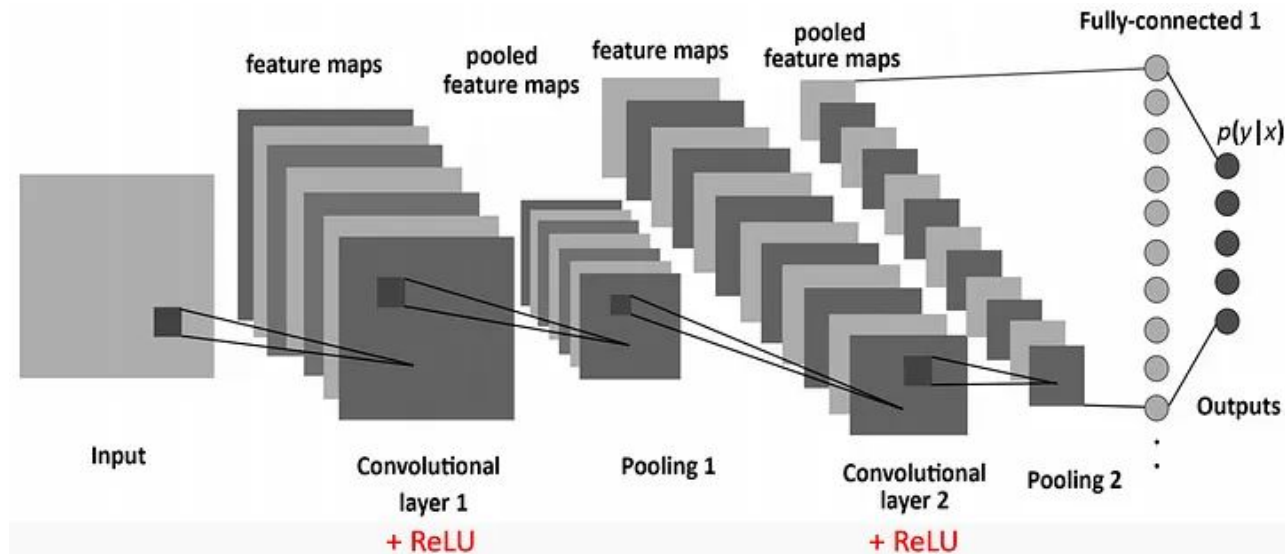
Pooling

- Feature maps preserve the location of features relative to the input image (**equivariance**).
- For classification tasks, we often want the network to be **invariant** to small translations.
- **Example:** Recognizing a face depends on the relative positions of eyes, nose, and mouth, not their exact pixel locations.

Role of Pooling in Achieving Invariance

- Pooling is applied to feature maps to make the network invariant to small translations of features.
- Helps reduce sensitivity to small shifts in feature locations without affecting classification.
- Unlike convolution, pooling does not learn weights; it applies a fixed function to its inputs.

Pooling - Example



<https://towardsdatascience.com/simple-introduction-to-convolutional-neural-networks-cdf8d3077bac>

An example CNN with two convolutional layers, two pooling layers, and a fully connected layer which decides the final classification of the image into one of several categories.

Each feature map in the CNN undergoes pooling independently, preserving hierarchical feature extraction.

Pooling

Max-Pooling: Most Common Pooling Method

- Selects the maximum value from the receptive field.
- Helps preserve strongest activations, ensuring key features remain detected.
- Uses a stride equal to the filter size, meaning no overlap between pooling regions.

3	5	4	6
1	1	9	4
7	10	9	5
12	2	9	4



5	9
12	9

Pooling for Dimensionality Reduction

- Down-samples feature maps, reducing computational cost.
- Larger strides in convolutional layers can also help down-sample the representation.

Other Pooling Methods

- Average Pooling: Computes the average of values in the pooling region.
- Introduces local translation invariance but retains more spatial information.

Padding

If we apply convolutions on a normal image, the result will be down-sampled by an amount depending on the size of the filter.

This reduction may cause a loss of boundary information. What do we do if we don't want this to happen? **We can use padding.**

Padding essentially makes the feature maps produced by the filter kernels the same size as the original image.

- To preserve spatial size, we **pad** the image by adding extra pixels around its borders.
- This increases the feature map size, preventing excessive shrinkage.

Padding

Types of Convolution Based on Padding

- **Valid Convolution (No Padding):**
 - When $P=0$, the feature map is smaller than the input.
- **Same Convolution (Padding Applied):**
 - When $P=(M-1)/2$, the feature map size remains the same as the input.

Padding Strategies in Computer Vision

- Filters typically have odd sizes (e.g., 3×3 , 5×5) to maintain symmetry.
- Padding can be applied symmetrically on all sides of the image.

0	0	0	0	0	0
0	X_{11}	X_{12}	X_{13}	X_{14}	0
0	X_{21}	X_{22}	X_{23}	X_{24}	0
0	X_{31}	X_{32}	X_{33}	X_{34}	0
0	X_{41}	X_{42}	X_{43}	X_{44}	0
0	0	0	0	0	0

Choosing Padding Pixel Values

- A common approach is to set padding values to zero (zero-padding).
- Alternatively, the mean of the image pixel intensities can be subtracted first to normalize padding values.

Padding in Deeper CNN Architectures

- Helps retain edge features that would otherwise shrink away in deeper layers.
- Ensures consistent spatial dimensions for processing in subsequent convolutional layers.

Strided Convolutions

Convolutional Feature Map Size

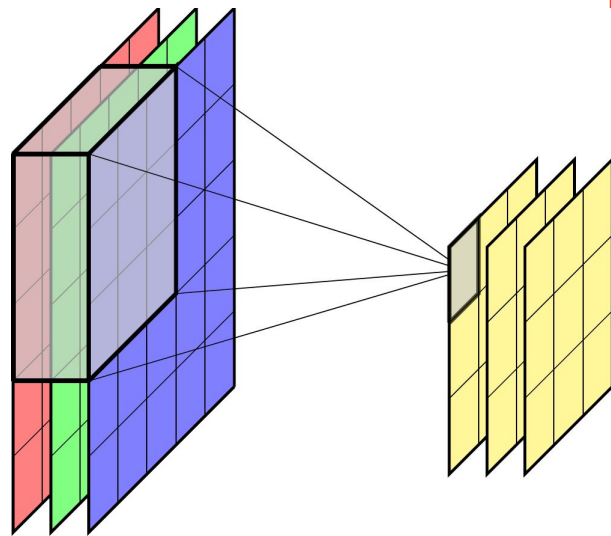
- In typical image processing, images have large pixel dimensions.
- Kernels are usually small relative to the image size $(M \ll J, K)$.
- If same padding is used, the feature map retains similar dimensions to the original image.

Reducing Feature Map Size with Strided Convolutions

- Instead of moving the filter one pixel at a time, we move it by a fixed step size S .
- This step size is called the **stride**.
- Larger strides result in smaller feature maps, reducing computational cost.

Multi-dimensional Convolutions

- Previously, we considered single-channel (grayscale) images.
- Color images have three channels: Red, Green, and Blue (RGB).
- A color image with $J \times K$ pixels and C channels is represented as a tensor with dimensionality: $J \times K \times C$
- Each pixel has multiple values corresponding to different color intensities.

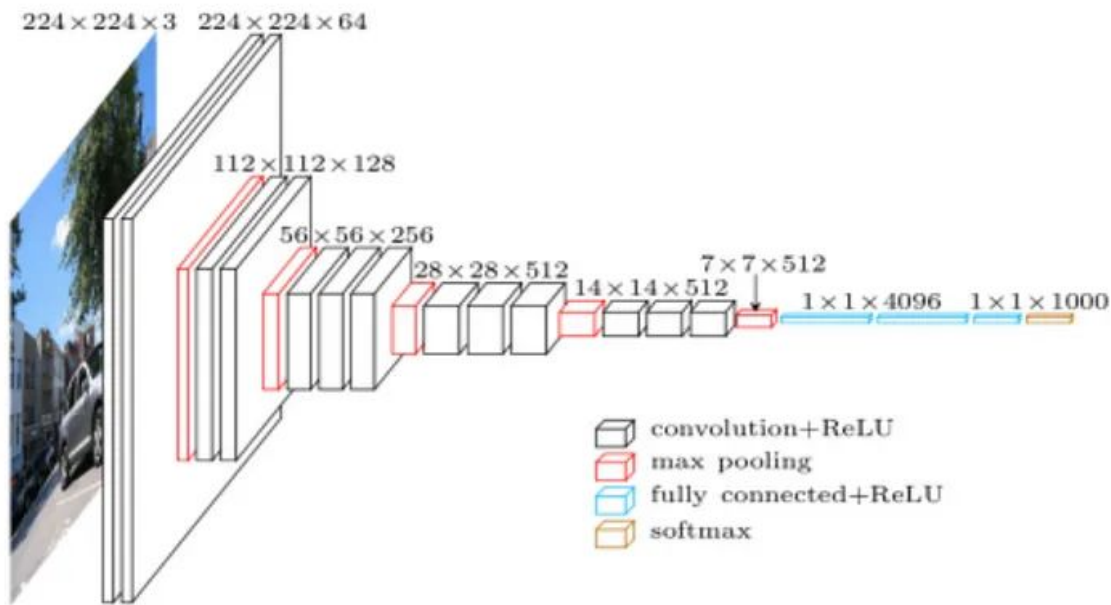


Filters for Multi-Channel Convolution

- Instead of a single $M \times M$ filter, we use a filter with dimensionality: $M \times M \times C$

Convolutional Neural Network Layers

There are three types of layers in a convolutional neural network: convolutional layer, pooling layer, and fully connected layer. Each of these layers has different parameters that can be optimized and performs a different task on the input data.



Convolutional Layers

Action

- Apply filters to extract features.
- Filters are composed of small kernels, learned.
- One bias per filter.
- Apply activation function on every value of the feature map.

Parameters

- Number of kernels.
- Size of kernels (Width and Height only, Depth is defined by input cube).
- Activation function.
- Stride.
- Padding.
- Regularization type and value.

I/O (Input/Output)

- **Input:** 3D cube, previous set of feature maps.
- **Output:** 3D cube, one 2D map per filter.

Pooling Layers

Action

- Reduce dimensionality.
- Extract maximum or average of a region.
- Sliding window approach.

Parameters

- Stride.
- Size of window/grid

I/O (Input/Output)

- **Input:** 3D cube, previous set of feature maps.
- **Output:** 3D cube, one 2D map per filter, reduced spatial dimensions.

Fully Connected Layers

Action

- Aggregate information from final feature maps.
- Generate final classification.

Parameters

- Number of nodes.
- Activation function:
 - a. Changes depending on the role of the layer.
 - b. If aggregating information, use **ReLU**.
 - c. If producing final classification, use something like **Softmax**.

I/O (Input/Output)

- **Input:** Flattened 3D cube, previous set of feature maps.
- **Output:** 3D cube, one 2D map per filter.

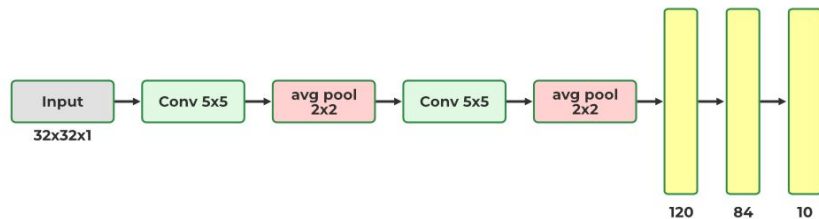


Example Network Architectures

Example Network Architectures

1. LeNet-5 (1998)

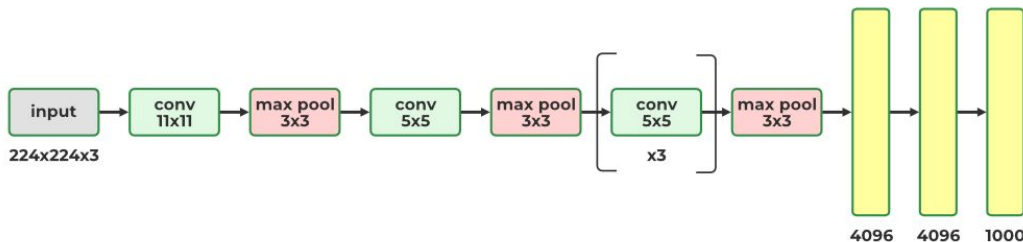
- **Application:** Handwritten digit recognition (MNIST).
- **Key Features:**
 - Uses small filters (5×5).
 - Introduced **pooling** for dimensionality reduction.
 - One of the first CNNs used for practical applications.



Example Network Architectures

2. AlexNet (2012)

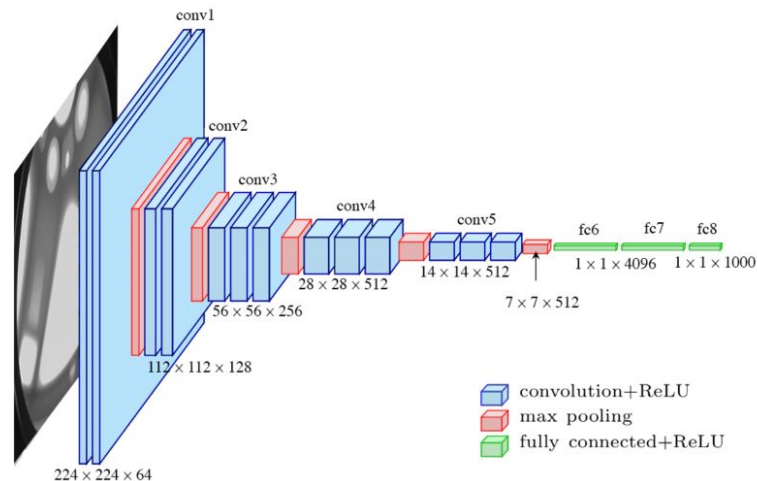
- **Application:** ImageNet Classification (1.2M images, 1000 classes).
- **Key Features:**
 - Used **ReLU activation** instead of sigmoid.
 - **Overlapping max pooling.**
 - **Dropout** to prevent overfitting.
 - Trained on **multiple GPUs**.



Example Network Architectures

3. VGG-16 (2014)

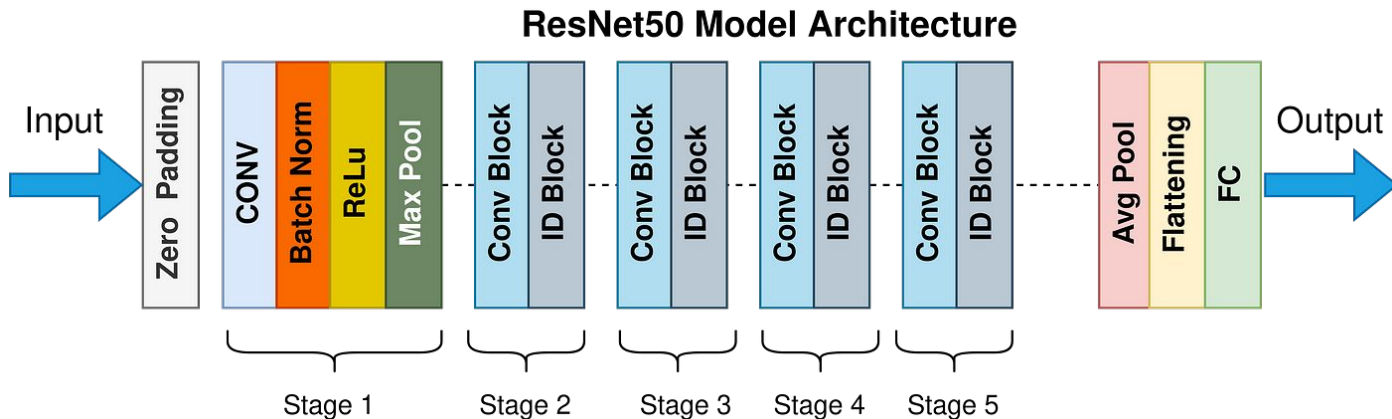
- **Application:** Image Classification (ImageNet).
- **Key Features:**
 - Deep network (16 layers).
 - Small 3×3 filters, improving feature extraction.
 - Computationally expensive.



Example Network Architectures

4. ResNet-50 (2015)

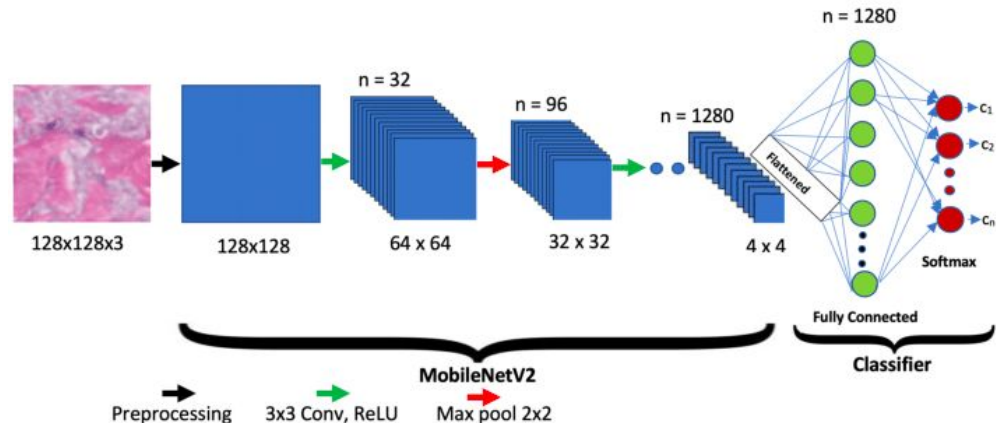
- **Application:** Deep learning classification tasks.
- **Key Features:**
 - **Residual Learning** enables training of very deep networks.
 - Skip connections allow deeper architectures to maintain gradient flow.
 - Outperforms previous networks on ImageNet.



Example Network Architectures

5. MobileNet (2017)

- **Application:** Mobile and embedded vision applications.
- **Architecture:**
 - Uses **depthwise separable convolutions** to reduce computation.
 - Smaller number of parameters compared to ResNet/VGG.
 - Efficient for edge devices and mobile applications.
- **Key Features:**
 - Optimized for low-power devices.
 - Reduces number of computations while maintaining accuracy.

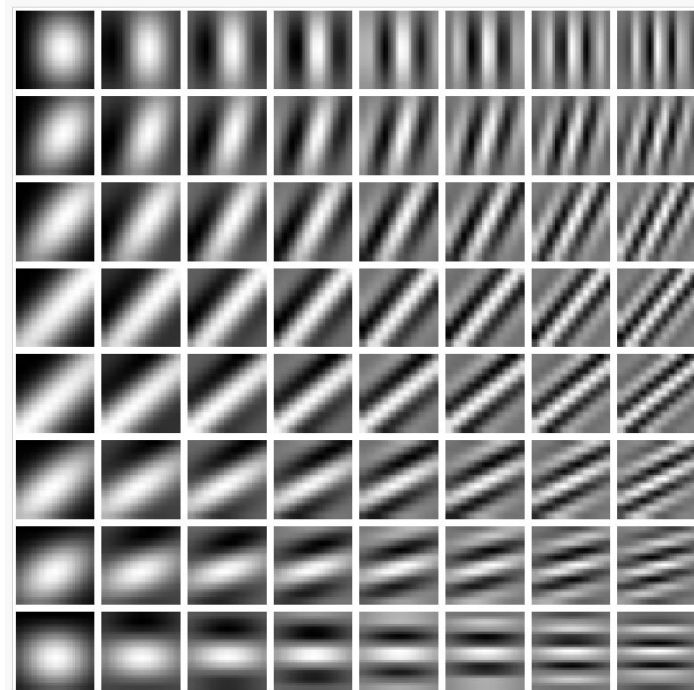


A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of parallel lines that curve and branch out.

Visualizing Trained CNNs

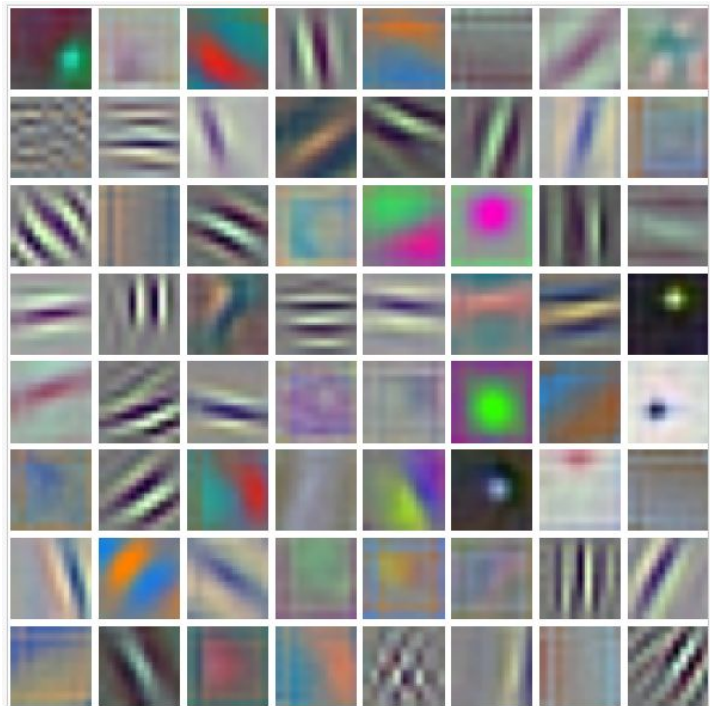
Visual Cortex

- CNNs were inspired by how the human visual system processes images, particularly research on the visual cortex by Hubel & Wiesel (1959).
- Simple cells in the brain detect edges and orientations, which can be mathematically modeled using Gabor filters, forming the foundation for feature extraction in CNNs.
- Complex cells combine inputs from multiple simple cells, enabling invariance to position and movement, similar to pooling layers in CNNs.
- Higher-order neurons recognize complex patterns, analogous to deeper layers in CNNs that detect meaningful structures, from edges to objects.



Examples of Gabor filters
(Bishop, 2024)

Visualizing Trained Filters

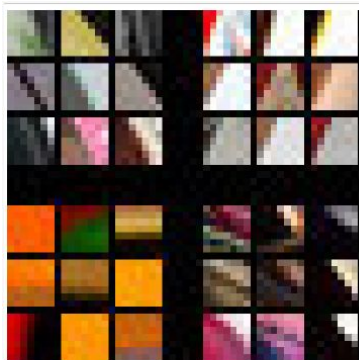


Examples of Learned Filters
from First Layer of AlexNet
(Bishop, 2024)

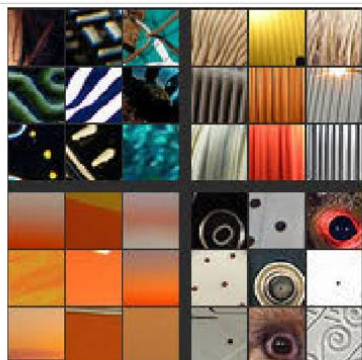
- Filters in the first convolutional layer of a deep CNN correspond to small patches in the original input image, similar to how early visual processing occurs in the brain.
- The first convolutional layer's filters show a strong resemblance to Gabor filters, similar to those found in biological vision systems.
- Despite the similarity, CNNs are not direct models of the brain but share statistical properties that make them effective for image understanding.
- Higher layers in CNNs are harder to interpret, as they do not process image patches directly but rather features extracted from previous layers, requiring different visualization techniques.

Visualizing Trained Filters

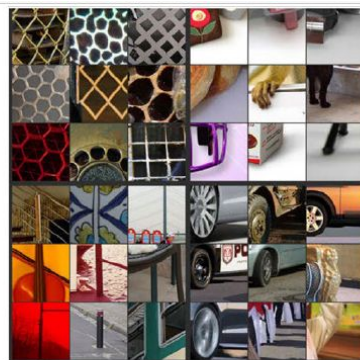
- Optimization techniques can modify input images to maximize activation for a particular class, rather than selecting high-activation images manually.
- Unconstrained optimization of activations can create high-frequency patterns that are difficult to interpret, requiring regularization.



Layer 1



Layer 2



Layer 3

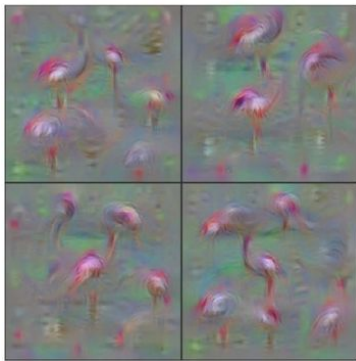


Layer 5

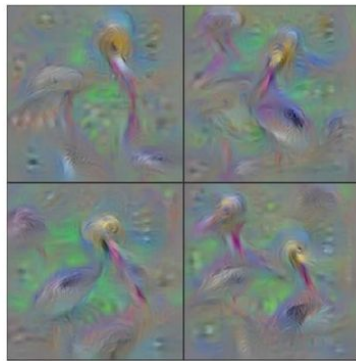
Examples of image patches that produce the strongest activation in the hidden units in a network having five convolutional layers trained on ImageNet data
(Bishop, 2024)

Visualizing Trained Filters

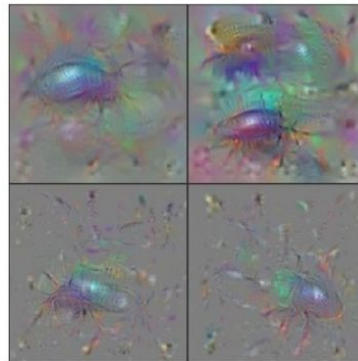
- **Regularization techniques**, such as blurring and clipping weak contributions, help refine optimized images to resemble natural patterns.
- **Adversarial training** is related to this process, as optimizing activations without constraints can lead to exaggerated and misleading feature representations.



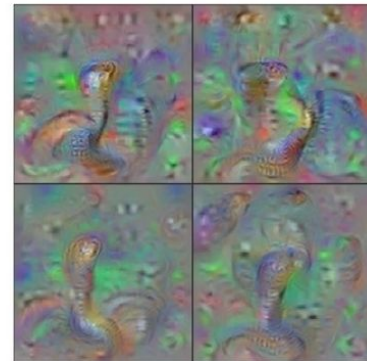
Flamingo



Pelican



Ground Beetle



Indian Cobra

Examples of synthetic images generated by maximizing the class probability (**using gradient ascent!**) with respect to the image pixel channel values for a trained convolutional classifier
(Bishop, 2024)

Saliency Maps – Grad-CAM

- Saliency maps identify the most important image regions for a given class by analyzing CNN activation patterns.
- Grad-CAM (Gradient-weighted Class Activation Mapping) computes the importance of each feature map in the **final convolutional layer** by averaging the gradients:

$$\alpha_k = \frac{1}{M_k} \sum_i \sum_j \frac{\partial a_c}{\partial a_{ij}^{(k)}}$$

M_k is the number of units in feature map k , and a_c activation for class c .

- The final weighted activation map is obtained by

$$L = \sum_k \alpha_k A^{(k)}$$

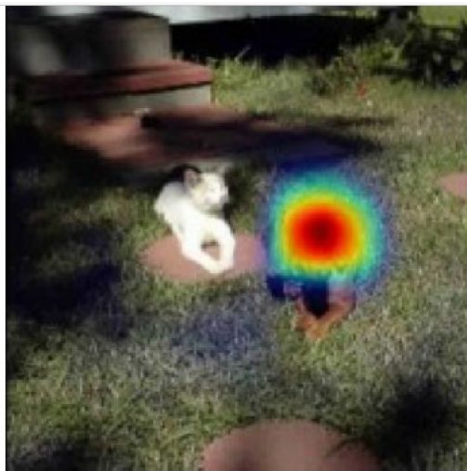
where $A^{(k)}$ is the activation map for feature k

Saliency Maps

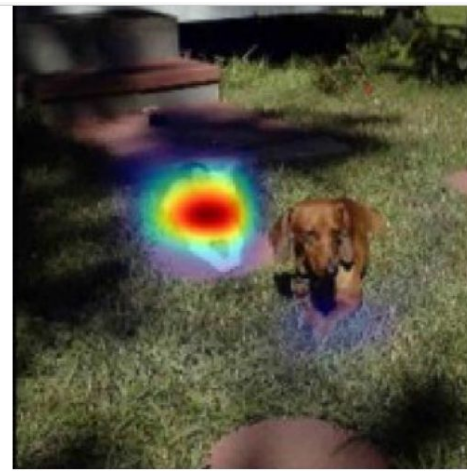
- The resulting heatmap preserves spatial localization lost in fully connected layers and can be overlaid on the original image to visualize key decision regions.



Original image



Saliency map for 'dog'



Saliency map for 'cat'

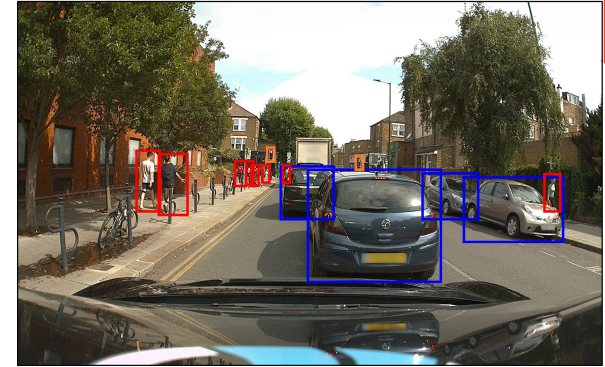
Saliency maps for the VGG-16 network with respect to the 'dog' and 'cat' categories
(Bishop, 2024)



Object Detection

Bounding Boxes

- CNNs are primarily designed for image classification, where an entire image is assigned to a single category (e.g., "cat" or "bicycle").
- CNNs can be adapted for other tasks beyond classification by leveraging their inductive biases, such as spatial hierarchy and feature extraction.
- Many images contain multiple objects belonging to different classes, requiring both detection and classification of each object.
- Object localization is crucial in computer vision applications, such as autonomous vehicles detecting pedestrians, road signs, and other vehicles.



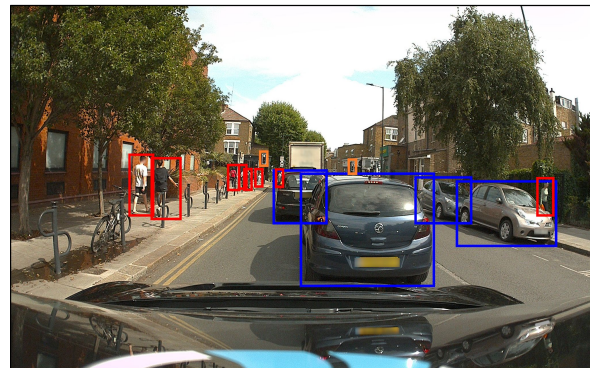
(Bishop, 2024)

Bounding Boxes

- A common approach for localization is using a bounding box, a rectangle that encloses the detected object and is defined by its center coordinates and dimensions:

$$b = (bx, by, bW, bH)$$

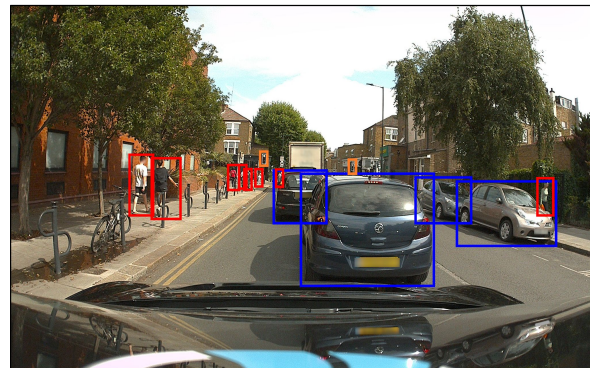
- Bounding box coordinates can be expressed in pixels or normalized values (where the top-left is (0, 0) (0,0) and the bottom-right is (1, 1) (1,1), allowing flexibility in different vision tasks.



(Bishop, 2024)

Bounding Boxes

- When an image contains a **single object** from a predefined set of C classes, a CNN typically has C output units with **softmax activation** for classification.
- Object localization can be achieved using **four additional outputs** trained to predict the bounding box coordinates: (bx, by, bW, bH)
- Since bounding box coordinates are **continuous values**, a **sum-of-squares error function** is commonly used for training these outputs.
- The YOLO (You Only Look Once) approach by Redmon et al. (2015) divides an image into a 7×7 grid, where each grid cell predicts object class and bounding box coordinates based on global image features.

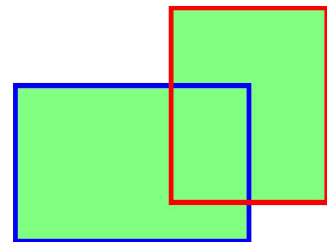


(Bishop, 2024)

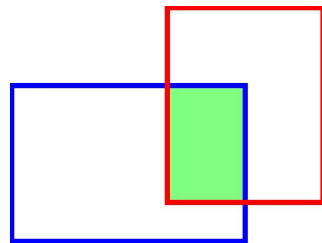
Intersection Over Union

- In image classification, network performance is measured using the log-likelihood of the true class labels over a test set.
- For object localization, performance is evaluated by comparing predicted bounding boxes to ground truth labels (e.g., human-annotated data).
- A key metric for measuring bounding box accuracy is Intersection-over-Union (IoU), which quantifies the overlap between predicted and ground truth boxes
- IoU is defined as:

$$\text{IoU} = \text{Area of Intersection} / \text{Area of Union}$$



area of union



area of intersection

Intersection-over-Union (IoU) ranges from 0 to 1, where higher values indicate better overlap between predicted and ground truth bounding boxes. A prediction is typically considered correct if the IoU exceeds a threshold, commonly set at 0.5

A series of white lines on the left side of the slide, consisting of several horizontal lines and a few diagonal lines that create a stepped, architectural effect.

Image Segmentation

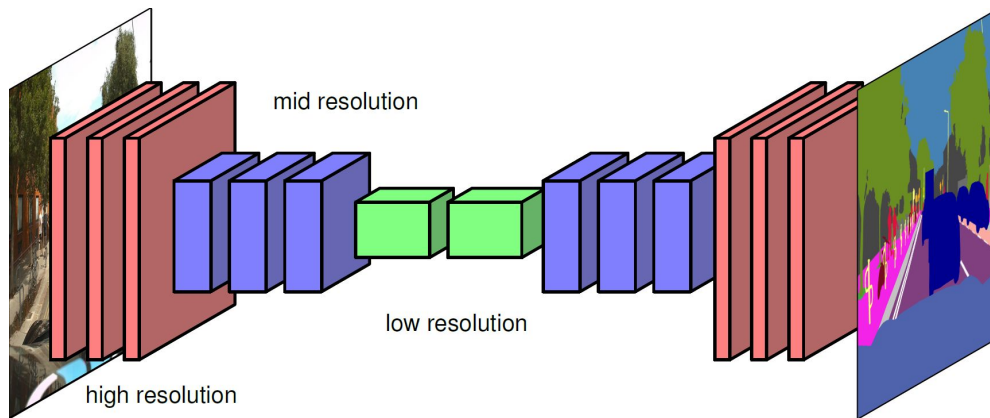
Convolutional Segmentation

- Semantic segmentation assigns a class label to each pixel in an image
- The output of a segmentation model has the same spatial dimensions as the input image, allowing predictions to be represented as a pixel-wise classification map.
- The final segmentation result is often visualized by coloring each pixel according to the class with the highest probability, creating an intuitive representation of detected regions.

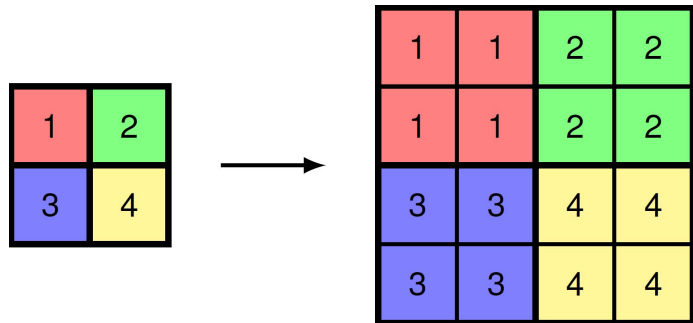


Up-Sampling

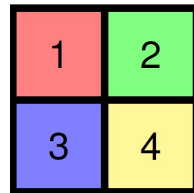
- Most CNNs use down-sampling (via strided convolutions and pooling) to reduce feature map size, keeping computations manageable while extracting high-level features.
- In semantic segmentation, this down-sampling must be reversed to recover the original image resolution while maintaining learned feature representations.
- Up-sampling techniques serve as the inverse of pooling, increasing the spatial dimensions of feature maps to reconstruct fine-grained segmentation details.



Up-Sampling



(a)



(b)

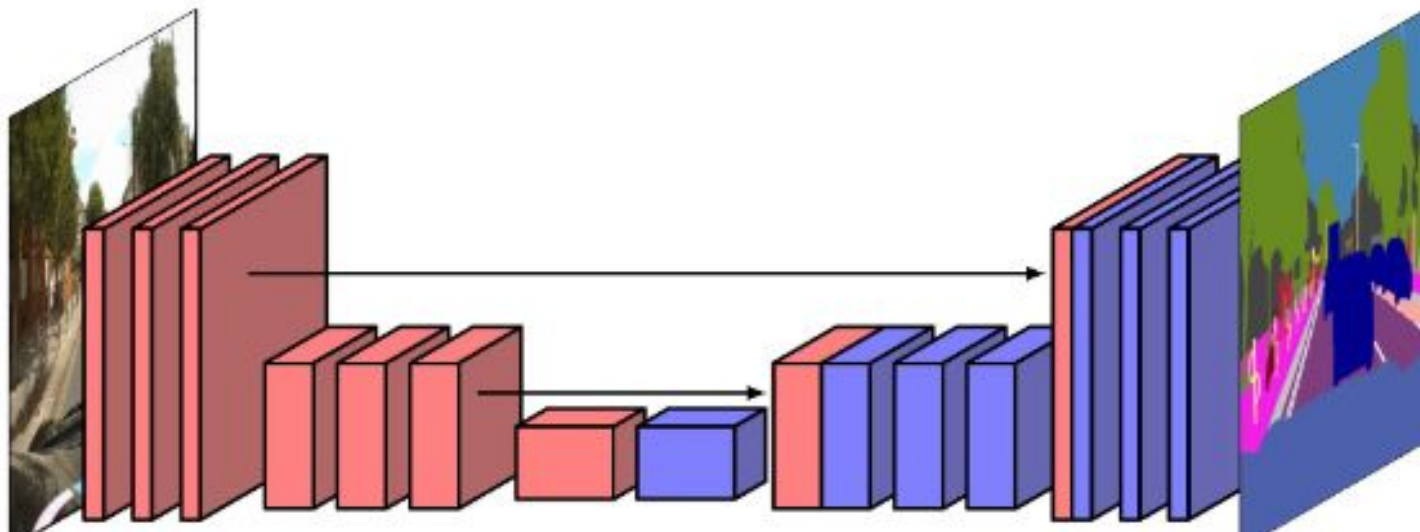
Illustration of unpooling operations showing (a) an analogue of average pooling and (b) an analogue of max-pooling

(Bishop, 2024)

The U-net Architecture

- **Down-sampling in CNNs** increases the number of channels while reducing spatial resolution, which helps in image classification but discards positional information.
- **Semantic segmentation requires per-pixel classification**, making the loss of spatial information problematic.
- U-Net architecture (Ronneberger, Fischer, and Brox, 2015) addresses this by incorporating both **down-sampling and up-sampling paths** to retain spatial details.
- The U-Net design is named after **its U-shaped structure**, enabling effective segmentation by combining low-level spatial features with high-level contextual information.

The U-net Architecture



(Bishop, 2024)

The background is a solid orange color. On the left side, there are several white, stylized lines that resemble a circuit board or a series of parallel paths. These lines start from the left edge and extend towards the center, with some branching out. The text 'Transfer Learning' is centered on the right side of the slide. The word 'Transfer' is on the top line, and 'Learning' is on the bottom line. A thin white horizontal line is positioned directly beneath the word 'Learning'.

Transfer Learning

Using existing architectures

- Transfer Learning leverages the knowledge from a pre-trained model (usually trained on a large dataset) to solve a new, related task with limited data.
- Combined Benefit: Using a CNN that's already been trained on a large and diverse dataset allows you to quickly and efficiently build a robust model for your specific problem, reducing training time and often improving performance.
 - We will play with existing architectures in our projects!